

# Lab 8: Throughput Engineering — Bulk Import, Batching & the Schema Pattern Playbook (60 min)

This is the **performance capstone of Day 2**. Everything you measure here is a number you can quote in a design review. Pairs with the [Schema Patterns Playbook](#).

## Learning Objectives

By the end of this lab you will be able to:

- Measure and rank the four ways of getting rows into CockroachDB: single-row `INSERT`, multi-row `INSERT`, `COPY`, and `IMPORT INTO`
- Find the **batch-size knee** for your workload instead of guessing at 100 or 1000
- Pre-split a table with `ALTER TABLE ... SPLIT AT` and prove the import got faster
- Run the same write workload against four PK designs and produce a rows/sec table
- Replace a single-row counter with a **Sharded Counter** and measure the retry rate collapse
- Run an online schema change under live write load and quantify the impact
- Use `cockroach workload` to find the concurrency knee where throughput plateaus but p99 explodes
- Size a connection pool from measured numbers rather than folklore

## Prerequisites

- **Docker Desktop** (or Docker Engine) running — there is no `cockroach` binary to install
- Labs 3 (schema design) and 6 ( `EXPLAIN ANALYZE` ) — you'll reuse both
- `psql` on `PATH` for the `COPY` part (optional; a fallback is provided)

## Setup

The 3-node lab cluster — three nodes is the minimum that makes distribution visible.

```
scripts/crdb up          # start the 3-node cluster (skip if it is already running)
scripts/crdb sql         # open a SQL shell
```

Everything runs in Docker — see [Lab 1](#) for the cluster layout. Use `scripts/crdb.sh` on macOS, Linux, and **WSL** — WSL is Linux, so the shell script is the right one there. Only a native Windows shell (`cmd` or PowerShell) needs `scripts\crdb.bat`.

```
CREATE DATABASE throughput;
USE throughput;
SET sql_safe_updates = off;
\set show_times
```

Several parts of this lab drive the cluster from a **second terminal**. Export the connection URL there (the shell's `-e` commands below run in the `throughput` database explicitly, so the shell you opened above can stay in it):

```
export CRDB_URL='postgresql://root@localhost:26257/throughput?sslmode=disable'
scripts/crdb sql -e "SELECT 1;" # confirm before continuing
```

**Keep a results table open.** Every part of this lab produces a number. Record it — the final deliverable is the filled-in table at the end, not the individual commands.

## Tasks

### Part A: Four Ways to Load 100,000 Rows (15 min)

The target schema is deliberately boring so the measurement is about the *method*, not the design:

```
CREATE TABLE load_test (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant      INT NOT NULL,
  amount      DECIMAL(12,2) NOT NULL,
  note        STRING NOT NULL,
  created     TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

#### A1. Single-row inserts (*the baseline you never want in production*)

Because a per-row round trip is the point, run this from the shell, not from SQL:

```
# In a second terminal.
time (for i in $(seq 1 2000); do
  echo "INSERT INTO throughput.load_test (tenant, amount, note) VALUES (1, 9.99, 'row');"
done | scripts/crdb sql)
```

2000 rows — not 100,000. Extrapolate:

```
-- rows/sec for the single-row method
SELECT 2000 / <seconds_from_time> AS rows_per_sec;
```

**Why so slow?** Each statement is its own implicit transaction: a full Raft round trip per row, plus a network round trip per row. You are measuring latency, not throughput.

## A2. Multi-row INSERT (batched)

```
TRUNCATE load_test;

-- 100,000 rows in ONE statement, generated server-side
INSERT INTO load_test (tenant, amount, note)
SELECT (g % 50), (g % 1000)::DECIMAL / 100, 'batched row ' || g
FROM generate_series(1, 100000) g;
```

Record the elapsed time. This is one statement, one transaction, and CockroachDB pipelines the writes across ranges.

## A3. COPY

```
# Generate a CSV once
python3 - <<'PY'
import csv, random, uuid
with open('/tmp/load_test.csv', 'w', newline='') as f:
    w = csv.writer(f)
    for i in range(100_000):
        w.writerow([str(uuid.uuid4()), i % 50, round(random.random()*1000, 2),
                    f'copy row {i}', '2024-01-01 00:00:00+00'])
PY

time psql "$CRDB_URL" -c "TRUNCATE throughput.load_test" \
-c "\copy throughput.load_test (id, tenant, amount, note, created) FROM '/tmp/
load_test.csv' CSV"
```

No `psql` ? The built-in shell does **not** implement `\copy`, but `COPY ... FROM STDIN` works when the data is piped in after the statement:

```
time { echo "COPY throughput.load_test (id, tenant, amount, note, created) FROM STDIN
CSV;"; cat /tmp/load_test.csv; echo '\.'; } | scripts/crdb sql
```

## A4. IMPORT INTO

`IMPORT INTO` bypasses the SQL layer entirely and writes SSTables directly into the storage layer. It takes the table **offline** for the duration.

```
TRUNCATE load_test;
```

```
# The CSV is on your machine; the node is in a container and cannot see it.
# Copy it in first, then upload it to the cluster's userfile store.
scripts/crdb cp /tmp/load_test.csv crdb1:/tmp/load_test.csv
scripts/crdb run userfile upload /tmp/load_test.csv /lab8/load_test.csv --insecure
```

**This two-step is the containerised version of "put the file where the database can read it".** In production the same problem is solved by cloud storage — `IMPORT INTO ... CSV DATA ('s3://...')` — which is why real bulk loads point at S3/GCS rather than a local path.

```
IMPORT INTO load_test (id, tenant, amount, note, created)
CSV DATA ('userfile:///lab8/load_test.csv');
```

## A5. Record the results

| Method                         | Rows                 | Elapsed | Rows/sec | Table online during load?        |
|--------------------------------|----------------------|---------|----------|----------------------------------|
| Single-row <code>INSERT</code> | 2,000 (extrapolated) |         |          | <input type="checkbox"/>         |
| Multi-row <code>INSERT</code>  | 100,000              |         |          | <input type="checkbox"/>         |
| <code>COPY</code>              | 100,000              |         |          | <input type="checkbox"/>         |
| <code>IMPORT INTO</code>       | 100,000              |         |          | <input type="checkbox"/> offline |

**The decision rule.** `IMPORT INTO` for one-time loads and migrations where downtime on that table is acceptable. `COPY` for large loads that must stay online. Multi-row `INSERT` for application-driven ingest. Single-row `INSERT` only when the row arrives alone.

## Part B: Find the Batch-Size Knee (10 min)

"Batch your inserts" is useless advice without a number. Find yours.

```
TRUNCATE load_test;

-- Run each of these and record the elapsed time.
-- Total rows is constant (50,000); only the rows-per-statement changes.

-- batch = 1000 (50 statements)
INSERT INTO load_test (tenant, amount, note)
SELECT g % 50, 1.00, 'b' FROM generate_series(1, 1000) g; -- repeat 50x

-- Easier: drive the sweep from one script
```

```

for BATCH in 1 10 100 1000 5000 10000; do
  STMTS=$((50000 / BATCH))
  START=$(date +%s.%N)
  for i in $(seq 1 $STMTS); do
    echo "INSERT INTO throughput.load_test (tenant, amount, note) SELECT g % 50, 1.00, 'b'
        FROM generate_series(1, $BATCH) g;"
  done | scripts/crdb sql >/dev/null 2>&1
  END=$(date +%s.%N)
  echo "batch=$BATCH elapsed=$(echo "$END - $START" | bc)s rows/sec=$(echo "50000 / ($END
    - $START)" | bc)"
  scripts/crdb sql -e "TRUNCATE throughput.load_test" >/dev/null
done

```

| Batch size | Elapsed | Rows/sec | Notes                     |
|------------|---------|----------|---------------------------|
| 1          |         |          | latency-bound             |
| 10         |         |          |                           |
| 100        |         |          |                           |
| 1,000      |         |          | usually near the knee     |
| 5,000      |         |          |                           |
| 10,000     |         |          | watch for txn size limits |

**What you should see:** steep gains from 1 → 100, a knee somewhere around 500–2,000, then flat or *worse* past that. Very large batches hold more intents open, raise contention and retry cost, and risk hitting `kv.transaction.max_intents_bytes`. When a batch fails, you retry the *whole* batch — the knee is about failure blast radius as much as it is about speed.

## Part C: Pre-Split Before a Bulk Load (*Playbook #10*) (10 min)

A new table is one range. A 100M-row import into one range means every write goes to one leaseholder until the splitter catches up.

### 1. Un-split baseline:

```

CREATE TABLE seq_import (id INT PRIMARY KEY, payload STRING);

SELECT count(*) AS ranges_before FROM [SHOW RANGES FROM TABLE seq_import];

INSERT INTO seq_import
SELECT g, repeat('x', 200) FROM generate_series(1, 200000) g;

SELECT count(*) AS ranges_after FROM [SHOW RANGES FROM TABLE seq_import];

```

Record the elapsed time and the range count.

### 2. Pre-split version:

```
CREATE TABLE seq_import_split (id INT PRIMARY KEY, payload STRING);

ALTER TABLE seq_import_split SPLIT AT
  SELECT g * 10000 FROM generate_series(1, 19) g;

SELECT count(*) AS ranges_before FROM [SHOW RANGES FROM TABLE seq_import_split];

INSERT INTO seq_import_split
SELECT g, repeat('x', 200) FROM generate_series(1, 200000) g;
```

### 3. Compare leaseholder spread:

```
SELECT lease_holder, count(*) AS ranges
FROM [SHOW RANGES FROM TABLE seq_import_split WITH DETAILS]
GROUP BY lease_holder ORDER BY lease_holder;
```

### 4. Release the manual splits when the load is done — otherwise the splits are pinned forever and the cluster can't merge them back:

```
ALTER TABLE seq_import_split UNSPLIT ALL;
```

**Sizing the split points.** One split per ~512 MiB of expected data, or one per node × a small multiple (3–10), whichever is larger. You are seeding the allocator, not micro-managing it.

## Part D: Four PK Designs Under Concurrent Write Load (15 min)

Lab 3 showed you the *shape* of a hotspot. Now put a number on it.

### 1. Create the four tables:

```
CREATE TABLE pk_serial (
  id SERIAL PRIMARY KEY, tenant INT, payload STRING, created TIMESTAMPTZ DEFAULT now());

CREATE TABLE pk_uuid (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(), tenant INT, payload STRING,
  created TIMESTAMPTZ DEFAULT now());

CREATE TABLE pk_composite (
  tenant INT, id UUID DEFAULT gen_random_uuid(), payload STRING,
  created TIMESTAMPTZ DEFAULT now(), PRIMARY KEY (tenant, id));

CREATE TABLE pk_hash (
  tenant INT, created TIMESTAMPTZ DEFAULT now(), id UUID DEFAULT gen_random_uuid(),
  payload STRING,
  PRIMARY KEY (tenant, created, id) USING HASH WITH (bucket_count = 16));
```

### 2. Drive 8 concurrent writers at each table and time the whole run:

```

run_writers() {
  TABLE=$1; ROWS_PER_WRITER=2000; WRITERS=8
  START=$(date +%s.%N)
  for w in $(seq 1 $WRITERS); do
    ( for i in $(seq 1 $((ROWS_PER_WRITER / 100))); do
      echo "INSERT INTO throughput.$TABLE (tenant, payload) SELECT g % 50, 'p' FROM
      generate_series(1,100) g;"
    done | scripts/crdb sql >/dev/null 2>&1 ) &
  done
  wait
  END=$(date +%s.%N)
  TOTAL=$((ROWS_PER_WRITER * WRITERS))
  echo "$TABLE: $TOTAL rows in $(echo "$END - $START" | bc)s => $(echo "$TOTAL / ($END -
  $START)" | bc) rows/sec"
}

for t in pk_serial pk_uuid pk_composite pk_hash; do run_writers $t; done

```

### 3. Confirm the distribution difference:

```

SELECT 'pk_serial' AS t, count(*) AS ranges FROM [SHOW RANGES FROM TABLE pk_serial]
UNION ALL SELECT 'pk_uuid', count(*) FROM [SHOW RANGES FROM TABLE pk_uuid]
UNION ALL SELECT 'pk_composite', count(*) FROM [SHOW RANGES FROM TABLE pk_composite]
UNION ALL SELECT 'pk_hash', count(*) FROM [SHOW RANGES FROM TABLE pk_hash];

```

### 4. Check the hot-range signature while a run is in flight:

```

-- SQL gives you distribution, not traffic:
SELECT table_name, count(*) AS ranges, count(DISTINCT lease_holder) AS leaseholders
FROM [SHOW CLUSTER RANGES WITH TABLES, DETAILS]
WHERE database_name = 'throughput'
GROUP BY table_name ORDER BY ranges DESC;

```

For **traffic** per range, use **DB Console** → **Hot Ranges** in the left nav (or the `/_status/hotranges` endpoint) — per-range QPS is not exposed through SQL.

### 5. Now measure the cost of the design. Hash-sharding is not free — it makes ordered scans fan out across every bucket:

```

EXPLAIN ANALYZE
SELECT * FROM pk_hash
WHERE tenant = 7 AND created > now() - INTERVAL '1 hour'
ORDER BY created DESC LIMIT 100;

EXPLAIN ANALYZE
SELECT * FROM pk_composite WHERE tenant = 7 ORDER BY id LIMIT 100;

```

Compare the **rows read** and the number of spans scanned in each plan.

| PK design | Write rows/sec | Ranges | Ordered-scan cost | Use when            |
|-----------|----------------|--------|-------------------|---------------------|
| SERIAL    |                |        | cheapest          | never on a hot path |

| PK design    | Write rows/sec | Ranges | Ordered-scan cost     | Use when                            |
|--------------|----------------|--------|-----------------------|-------------------------------------|
| UUID         |                |        | no useful order       | writes dominate, point reads        |
| (tenant, id) |                |        | ordered per tenant    | multi-tenant, tenant-scoped queries |
| hash-sharded |                |        | fans out over buckets | time-ordered writes at high rate    |

## Part E: Sharded Counter vs Single-Row Counter (*Playbook #5*) (10 min)

### 1. The anti-pattern:

```
CREATE TABLE counter_single (name STRING PRIMARY KEY, n INT NOT NULL DEFAULT 0);
INSERT INTO counter_single VALUES ('page_views', 0);
```

### 2. The pattern:

```
CREATE TABLE counter_shards (
  name STRING,
  shard INT2,
  n INT NOT NULL DEFAULT 0,
  PRIMARY KEY (name, shard)
);
INSERT INTO counter_shards (name, shard, n)
SELECT 'page_views', g, 0 FROM generate_series(0, 15) g;
```

### 3. Record the retry baseline before you start:

```
SELECT sum((statistics->'statistics'->'maxRetries')::INT) AS max_retries
FROM crdb_internal.statement_statistics;
```

### 4. First, the trap — pick the shard the obvious way and count the result:

```
UPDATE counter_shards SET n = 0 WHERE name = 'page_views';
```

```
# 200 sequential increments, no concurrency at all
for i in $(seq 1 200); do
  echo "UPDATE throughput.counter_shards SET n = n + 1
    WHERE name = 'page_views' AND shard = (random()*16)::INT2;"
done | scripts/crdb sql >/dev/null 2>&1
```

```
SELECT sum(n) AS should_be_200 FROM counter_shards WHERE name = 'page_views';
```

**It will not be 200.** `random()` is a **volatile** function, and a volatile function in a `WHERE` predicate is evaluated **once per row scanned** — not once per statement. The statement therefore matches a random *number* of the 16 shard rows: often one, sometimes zero (the increment vanishes), sometimes three (it counts triple). See it directly:

```
SELECT count(*) FROM counter_shards WHERE shard = (random()*16)::INT2; -- run 8 times
```

You'll get a different count nearly every time. A scalar subquery ( `shard = (SELECT (random()*16)::INT2)` ) is no better.

**The fix: choose the shard where you have a stable value — in the application** — and bind it as a parameter. This is the single most common way a sharded counter is written wrong, and because the loss is silent it survives code review.

## 5. Now do it correctly, and hammer both designs with 16 concurrent bumpers:

```
bump_single() {
    START=$(date +%s.%N)
    for w in $(seq 1 16); do
        ( for i in $(seq 1 100); do
            echo "UPDATE throughput.counter_single SET n = n + 1 WHERE name = 'page_views';"
            done | scripts/crdb sql >/dev/null 2>&1 ) &
        done
    wait
    echo "single: 1600 increments in $(echo "$(date +%s.%N) - $START" | bc)s"
}

bump_sharded() {
    START=$(date +%s.%N)
    for w in $(seq 1 16); do
        ( for i in $(seq 1 100); do
            # the SHELL picks the shard – a literal by the time the server sees it
            echo "UPDATE throughput.counter_shards SET n = n + 1
                WHERE name = 'page_views' AND shard = $((RANDOM % 16));"
            done | scripts/crdb sql >/dev/null 2>&1 ) &
        done
    wait
    echo "sharded: 1600 increments in $(echo "$(date +%s.%N) - $START" | bc)s"
}

scripts/crdb sql -e \
    "UPDATE throughput.counter_single SET n = 0; UPDATE throughput.counter_shards SET n =
    0;"
bump_single
bump_sharded
```

Verify both counted every increment before you compare the times — a fast wrong answer is not a win:

```
SELECT (SELECT n FROM counter_single WHERE name = 'page_views') AS
       single_total,
       (SELECT sum(n) FROM counter_shards WHERE name = 'page_views') AS
       sharded_total;
-- both must equal 1600
```

## 6. Count the retries each design generated:

```
SELECT
  substring(metadata->'query', 1, 60) AS stmt,
  (statistics->'statistics'->'cnt')::INT AS executions,
  (statistics->'statistics'->'maxRetries')::INT AS max_retries,
  round((statistics->'statistics'->'svcLat'->'mean')::FLOAT * 1000, 2) AS mean_ms
FROM crdb_internal.statement_statistics
WHERE metadata->'query' LIKE '%counter%'
ORDER BY executions DESC;
```

## 7. Read the counter:

```
SELECT sum(n) FROM counter_shards WHERE name = 'page_views';
```

The read is now a 16-row scan instead of a 1-row point lookup. That's the trade: reads get marginally more expensive, writes stop serializing.

**Choosing the shard count.** Start at your peak concurrent writer count, round to a power of two, cap around 64. Too few shards leaves contention; too many makes the read scan wide and spreads the rows over more ranges than the workload needs.

## Results Summary — Fill This In

| Measurement                       | Your number | Rule of thumb              |
|-----------------------------------|-------------|----------------------------|
| Single-row insert rows/sec        |             | latency-bound, ~1/RTT      |
| Multi-row insert rows/sec         |             | 10–100× single-row         |
| <code>IMPORT INTO</code> rows/sec |             | fastest, table offline     |
| Batch-size knee                   |             | usually 500–2,000          |
| Pre-split speedup                 |             | biggest on sequential keys |
| Best PK design (writes)           |             | hash-sharded or UUID       |
| Worst PK design (writes)          |             | <code>SERIAL</code>        |
| Sharded counter speedup           |             | grows with concurrency     |
| Concurrency knee                  |             | pool size starts here      |

## Optional — If Time Allows

These parts are not required to complete the lab; they extend it by about 15 minutes. Do them if you finish early, or after class — the cluster and data from the core parts carry over.

### Part F: Online Schema Change Under Load (8 min)

#### 1. Start a background write stream:

```
( while true; do
    echo "INSERT INTO throughput.pk_uuid (tenant, payload) SELECT g % 50, 'live' FROM
        generate_series(1,200) g;"
done | scripts/crdb sql >/dev/null 2>&1 ) &
WRITER_PID=$!
```

#### 2. Add an index while writes are running:

```
CREATE INDEX ON pk_uuid (tenant, created DESC);
```

#### 3. Watch it as a job, not as a lock:

```
SELECT job_id, job_type, description, status, fraction_completed
FROM [SHOW JOBS] WHERE job_type IN ('SCHEMA CHANGE', 'NEW SCHEMA CHANGE')
ORDER BY created DESC LIMIT 5;
```

**Two schema-changer job types.** Modern CockroachDB runs most DDL through the *declarative* schema changer, which records jobs as **NEW SCHEMA CHANGE**. The legacy changer still handles some operations (e.g. **TRUNCATE**) and uses **SCHEMA CHANGE**. Filtering on only one of them is why people conclude "my **CREATE INDEX** didn't create a job" — match both.

#### 4. Measure the write-path cost of the new index:

```
SELECT
    (statistics->'statistics'->>'cnt')::INT AS executions,
    round((statistics->'statistics'->'svcLat'->>'mean')::FLOAT * 1000, 2) AS mean_ms
FROM crdb_internal.statement_statistics
WHERE metadata->'query' LIKE 'INSERT INTO pk_uuid%';
```

Every secondary index multiplies write amplification. Two indexes on a table means each **INSERT** writes three key-value entries, and each one is a Raft proposal.

#### 5. Stop the writer:

```
kill $WRITER_PID 2>/dev/null; pkill -f "generate_series(1,200)" 2>/dev/null
```

**The rule that matters in a deploy:** `CREATE INDEX`, `ADD COLUMN` with a default, and `DROP COLUMN` run as **background jobs** — non-blocking, resumable, and cancellable while traffic continues. `ALTER COLUMN TYPE` and adding a `NOT NULL` constraint to existing data require a validating backfill and are the ones to schedule carefully.

## Part G: The Concurrency Knee — cockroach workload (7 min)

Throughput plateaus long before latency does. Find the point where adding connections only adds queueing.

```
scripts/crdb run workload init kv --drop 'postgresql://root@crdb1:26257?sslmode=disable'

for C in 1 4 16 64 256; do
  echo "=== concurrency $C ==="
  scripts/crdb run workload run kv \
    --duration=30s --concurrency=$C \
    --read-percent=50 --max-rate=0 \
    'postgresql://root@crdb1:26257?sslmode=disable' 2>&1 | tail -4
done
```

| Concurrency | Throughput (ops/s) | p50 (ms) | p99 (ms) |
|-------------|--------------------|----------|----------|
| 1           |                    |          |          |
| 4           |                    |          |          |
| 16          |                    |          |          |
| 64          |                    |          |          |
| 256         |                    |          |          |

Plot throughput and p99 against concurrency. The knee is the last concurrency level where throughput still climbs meaningfully. Past it you are buying latency with no throughput.

**Pool sizing from these numbers.** Little's Law:  $\text{concurrency} = \text{throughput} \times \text{latency}$ . If the knee is 64 concurrent statements at 8 ms mean latency, a single app instance sustaining 2,000 QPS needs  $2000 \times 0.008 \approx 16$  connections — not 200. Size the pool to the knee divided by the number of app instances, then add a small headroom margin. An oversized pool does not add throughput; it moves the queue from your app into the database, where it is harder to see and more expensive to drain.

## Cleanup

```
USE defaultdb; -- the shell refuses to drop the database you are still in
DROP DATABASE throughput CASCADE;
DROP DATABASE IF EXISTS kv CASCADE;
```

```
rm -f /tmp/load_test.csv
```

Then `\q` to leave the SQL shell.

The cluster keeps running between labs — that is the point of it being persistent. To wipe everything and start fresh at any time:

```
scripts/crdb reset
```

## Lab 8 Deliverables

□ **Ingest ranking**: four load methods measured, with a stated decision rule for each □ **Batch knee**: an actual number for this schema, not a guess □ **Pre-split**: range count and load time before/after □ **PK bake-off**: rows/sec for four designs plus the ordered-scan cost each one pays □ **Sharded counter**: retry count and throughput vs the single-row version, *and* the volatile-predicate trap observed and fixed □ **Online schema change**: index added under live write load, job observed, write cost measured □ **Concurrency knee**: throughput/p99 curve and a pool size derived from it

## Challenge Exercises

1. **Beat IMPORT INTO**. Pre-split `load_test` on the UUID keyspace before the import ( `SPLIT AT VALUES ('11111111-...', '22222222-...', ...)` ). Does the import get faster? Why is the effect smaller than it was for the sequential-key table?
2. **Find the write-amplification curve**. Add indexes to `pk_uuid` one at a time (0, 1, 2, 4) and re-measure insert throughput at each step. Plot rows/sec against index count. What is the marginal cost of index number four?
3. **Break the batch**. Set `SET CLUSTER SETTING kv.transaction.max_intents_bytes = '1MiB'`, then find the batch size that starts failing. What error do you get, and what should the application do with it?
4. **Re-run Part D on more nodes**. Add `crdb5 ... crdb9` to `docker/labs.yml` (copy the `crdb4` block, bump the ports) and re-run. Which PK design's advantage grows with node count, and which one's shrinks?

## Reference

| Command                                            | Purpose                               |
|----------------------------------------------------|---------------------------------------|
| <code>IMPORT INTO t (cols) CSV DATA ('...')</code> | Fastest bulk load; table goes offline |
| <code>\copy t FROM 'file' CSV</code>               | Fast bulk load; table stays online    |
| <code>ALTER TABLE t SPLIT AT SELECT ...</code>     | Seed ranges before a bulk load        |

| Command                                                                          | Purpose                                                      |
|----------------------------------------------------------------------------------|--------------------------------------------------------------|
| <code>ALTER TABLE t UNSPLIT ALL</code>                                           | Release manual splits so ranges can merge                    |
| <code>scripts/crdb cp &lt;file&gt; crdb1:/tmp/</code>                            | Copy a host file into a node container                       |
| <code>scripts/crdb run userfile upload</code>                                    | Stage a file in the cluster for <code>IMPORT</code>          |
| <code>scripts/crdb run workload run kv --concurrency=N</code>                    | Throughput/latency curve                                     |
| <code>crdb_internal.statement_statistics</code>                                  | Executions, retries, mean latency per statement              |
| Volatile functions ( <code>random()</code> , <code>now()</code> ) in a predicate | Re-evaluated <b>per row</b> — never use them to select a row |
| <code>SHOW RANGES FROM TABLE t WITH DETAILS</code>                               | Range count, size, leaseholder placement                     |
| <code>SHOW JOBS</code>                                                           | Watch online schema changes progress                         |